SECURING APACHE

Part—10

Right from Part 1 of this series, we've covered the major types of attacks being done on Web applications—and their security solutions. In this article, I will reveal the tremendous capabilities of the Apache *mod_security* module, covering just a small part of what it can do.

From the development perspective, implementing security against the many attacks on Web apps doesn't just require extra coding and stronger validation, but often also results in complex and messy code, which may sometimes cause yet another security loophole. Security is often compared to a football game, where success requires the defence to quickly adapt, outrun, and outplay the attackers. Such a dynamic defence cannot properly survive in complex and messy code. Here, Web application firewalls come to the rescue—and what else is better than *mod_security*.

It is designed as an Apache module that adds intrusion-detection and prevention features to the Web server. In principle, it's similar to an IDS that analyses network traffic, but it works at the HTTP level, and really well, at that. This allows you to do things that are difficult in a classic IDS. This difference will become clearer when we examine several examples. The attack prevention feature stands between the client and server; if it finds a malicious payload, it can reject the request, performing any one of a number of built-in actions. Some of the features of *mod_security* are audit logging, access to any part of the request (including the body) and the response, a flexible regular expression-based rule engine, file-upload interception, real-time validation and also buffer-overflow protection.

How *mod_security* works

The module's functionality is divided into four main areas:

- Parsing: Security-conscientious parsers extract bits of each request and/or response and store them for use in the rules.
- Buffering: In a typical installation, both request and response bodies will be buffered so that the module usually sees complete requests (before they are passed to the application for processing), and complete responses (before they are sent to clients). Buffering is important, because it is the only way to provide reliable blocking.
- Logging: Allows you to record complete HTTP traffic, logging all response/request headers and bodies.
- Rule engine: Works on the data from the other components, to assess the transaction and take action, as necessary.